La méthode OWASP

Comment se tenir à jour ?

Prévenir plutôt que guérir… Quelques sites à surveiller :

- CISA (anciennement US-CERT)
- The Hacker News
- Zataz
- Reddit NetSec
- Next (anciennement Next INpact) (~payant)
- CERT-FR (ANSSI)
- Google Actu

> **Important**
> Vous êtes la première ligne d'informations !

Les grandes catégories

- **Confidentialité** : Risque de divulgation d'informations sensibles (ex. fuite de données personnelles).
- **Intégrité** : Risque de modification non autorisée de données ou de code (ex. injection SQL, XSS).
- **Disponibilité** : Risque de rendre un service indisponible (ex. attaque DDoS, ransomware).

Les grandes catégories de failles

- Les mots de passe (multi-facteurs, complexité, hachage).
- Les failles dans le code (injections, XSS, CSRF, etc.).
- Les failles dans les configurations (serveur, application, etc.).
- Le social engineering (le maillon faible, l'humain).

> **Le risque ?**
> Le risque est la multiplication des failles. Plus vous avez de failles, plus vous avez de risques. C'est là que la sécurité devient un enjeu majeur. Car plus le nombre de failles est important, plus **la surface d'attaque** est grande.

La surface d'attaque

La surface d'attaque est l'ensemble des points d'entrée d'un système qui peuvent être exploités par un attaquant pour compromettre la sécurité de ce système.

Plus la surface d'attaque est grande, plus le système est vulnérable.

Exemple

Une application Web avec un ensemble de failles (injection SQL, XSS, CSRF, etc.) a une grande surface d'attaque. Un attaquant peut exploiter ces failles pour compromettre la sécurité de l'application.

Un serveur mal configuré (ports ouverts, services non sécurisés, etc.) a une grande surface d'attaque. Un attaquant peut exploiter ces failles pour compromettre la sécurité du serveur.

Réduire la surface d'attaque est un enjeu majeur en sécurité informatique. Réduire la surface d'attaque implique de minimiser les points d'entrée.

L'authentification

Il est possible de sécuriser l'authentification de plusieurs manières :

- **Mots de passe** : Complexité, hachage, salage.
- **Authentification à plusieurs facteurs** : Double authentification, biométrie, OTP.

- **Sécurisation des mots de passe** : Bcrypt, Argon2, Scrypt.
- **Sécurisation des sessions** : JWT, Cookies sécurisés.
- **Authentification unique** : OAuth, OpenID (SSO, Single Sign-On).

Les risques du SSO

Le SSO (Single Sign-On) est une méthode d'authentification qui permet à un utilisateur de se connecter avec un seul identifiant et un seul mot de passe pour accéder à plusieurs applications. C'est une méthode très pratique, mais qui peut être dangereuse en cas de compromission (du mot de passe, de l'identifiant, de la session).

En effet, si un attaquant compromet un compte, il peut accéder à toutes les applications liées à ce compte.

Il est donc important de sécuriser le SSO avec des méthodes d'authentification à plusieurs facteurs (2FA, MFA).

Les mots de passe

Zoom sur les mots de passe :

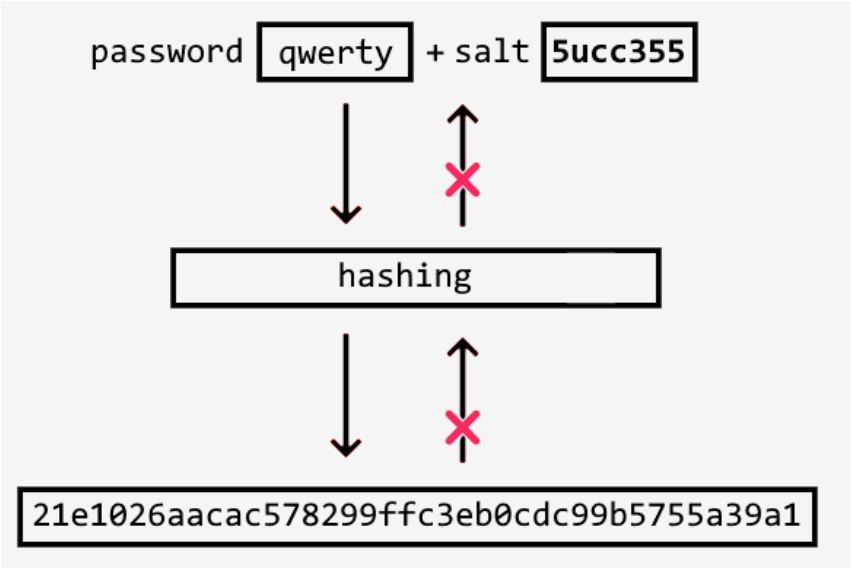
- Un mot de passe ne doit jamais être stocké en clair.
- Un mot de passe doit être haché (non réversible).
- Un mot de passe doit être salé (ajout d'une chaîne aléatoire).
- Un mot de passe seul n'est souvent pas suffisant (Double authentification).

Crypter ≠ Hasher

- **Crypter** : réversible (comme Canal+).
- **Hasher** : non réversible (comme le steak haché).

Principes de base

Avoir un mot de passe haché ne suffit pas. Il faut aussi le saler.



Le salage est une technique qui permet d'ajouter une chaîne aléatoire au mot de passe avant de le hacher. Le sel est unique pour chaque utilisateur et stocké avec le hash : deux mots de passe identiques donnent deux hash différents.

Des algorithmes : Le bcrypt

Le bcrypt est un algorithme de hachage qui :

- Intègre le sel.

- Intègre un coût (nombre d'itérations). Plus le coût est élevé, plus le hachage est long (et donc plus sécurisé).
- Est basé sur l'algorithme de chiffrement Blowfish (et non SHA-256).
- A des alternatives modernes : Argon2, Scrypt.

Résumé

Les mots de passe :

- Un mot de passe ne doit jamais être stocké en clair. Il doit être haché (non réversible) et salé (ajout d'une chaîne aléatoire).
- Le sel doit être unique par utilisateur. Celui-ci doit être stocké avec le hash, il sera utilisé également pour vérifier le mot de passe. Un secret global partagé par tous les utilisateurs existe aussi : on l'appelle un « poivre » (pepper), il vient en complément du sel, pas en remplacement.
- Le bcrypt est un algorithme de hachage qui intègre le sel, le coût et le hachage (basé sur Blowfish, et non SHA-256). Argon2 est recommandé comme alternative moderne (gagnant du Password Hashing Competition 2015).

Authentification à plusieurs facteurs

3 formes d'authentification :

- Mémorielle qui représente une chose que l'intéressé connaît (un secret),
- Matérielle qui se réfère à quelque chose qu'il possède (un objet),
- Corporelle qui utilise un trait physique de l'utilisateur (une biométrie).

Des **outils** :

- **Mot de passe :** Un mot de passe avec un niveau de sécurité suffisant (longueur, caractères spéciaux, majuscules, minuscules, chiffres).
- **Application :** OTP (One Time Password) : SMS, Google Authenticator, Authy.
- **Objet physique :** U2F (Universal 2nd Factor) : Clé USB, Yubikey.
- **Biométrie :** Empreinte digitale, Reconnaissance faciale.

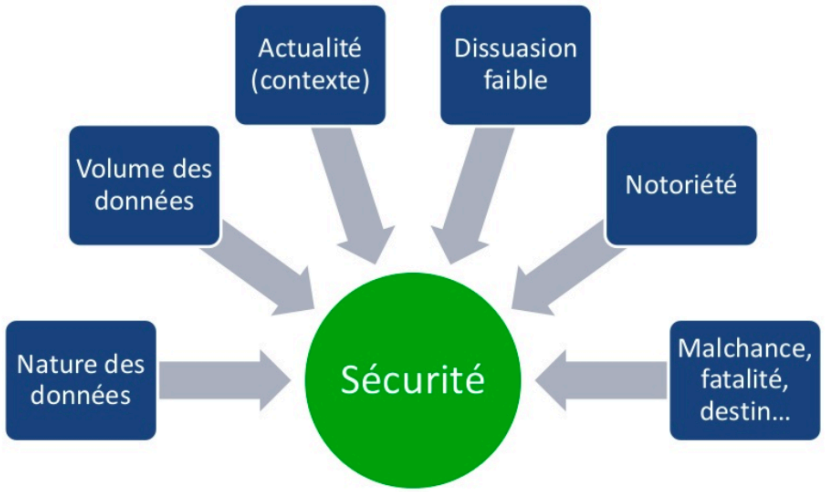
Mais surtout, l'authentification à plusieurs facteurs :

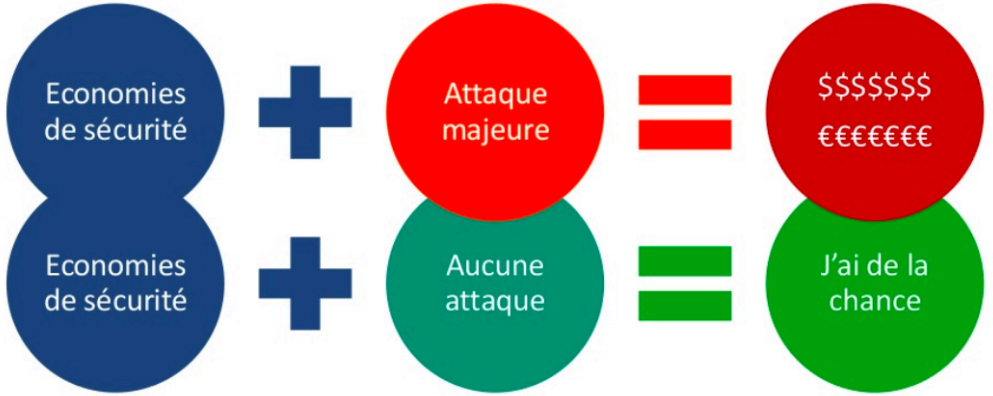
- Sécurise le compte en ajoutant une couche de sécurité supplémentaire au mot de passe.
- Repose sur un secret partagé entre la personne **physique** et le site.

Les impacts liés à la sécurité

La sécurité informatique dans une application, c'est un « équilibre » :

- Impact fonctionnel
- Limitation de l'expérience utilisateur (UX)
- Impact financier





Les types de failles

- L'humain (Social Engineering)
- D'accès (physique)
- Applicatif (Hack)
- L'argent (jusqu'à quel montant une personne donne l'information ?)

Le Social Engineering

Les gens sont souvent trop confiants. Il faut donc les former régulièrement à la sécurité.

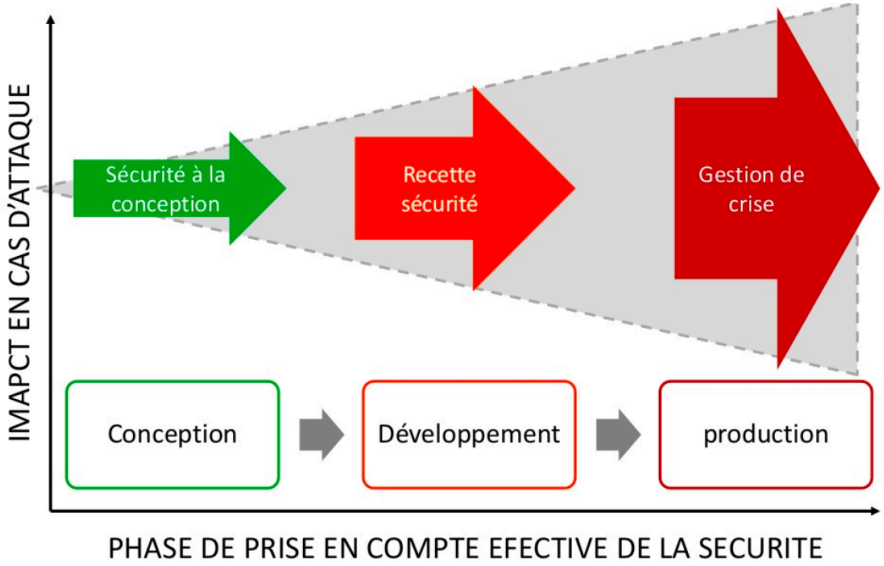
Deux exemples en vidéo :

Exemple SECTF à la DefCon	Call Recreation (@5min, @11min30)

Intégrer la sécurité à toutes les étapes

La sécurité c'est un état d'esprit à intégrer.

C'est **votre métier**.



L'observabilité

L'observabilité est un concept qui permet de mesurer et d'analyser le comportement d'un système. On parlera de traçabilité, de logs, de monitoring, de métriques, etc.

La traçabilité est un élément clé de la sécurité. Elle permet de savoir qui a fait quoi, quand et comment.

Supply Chain Attacks

Les Supply Chain Attacks sont des attaques ciblant les dépendances logicielles. Très courantes de nos jours. Les dépendances logicielles sont partout dans nos applications (nuget, composer, npm, pip, etc.), mais également sur nos serveurs (apt, yum, docker, etc.). Exemples : l'attaque SolarWinds en 2020, la backdoor découverte dans XZ Utils en 2024.

Concrètement, voici comment ça peut se dérouler :

- Une dépendance est compromise (par un attaquant ou un développeur malveillant).
- Un malware est injecté dans la dépendance.
- Le malware est distribué via la dépendance.
- Le malware s'exécute sur les systèmes des utilisateurs de la dépendance (serveur ou poste de travail en fonction du type de dépendance).

Vous êtes donc la cible ici d'une attaque pas directement de votre code, mais d'une dépendance que vous utilisez.

Ce type d'attaque est devenu si courant que depuis 2025, il a sa propre catégorie dans le Top 10 OWASP : **A03:2025 Software Supply Chain Failures** (voir plus bas).

Assurer la qualité

S'assurer d'une qualité continue du code avec :

- Des règles à connaître (OWASP).
- Des tests unitaires.
- Une analyse automatique du code (SonarQube).

L'IA et la sécurité

Les assistants IA produisent du code rapidement et en grande quantité. C'est un gain de temps, mais aussi un nouveau risque : ce code contient les mêmes failles que celui d'un humain, et on a tendance à lui faire davantage confiance.

Les risques :

- Beaucoup de code produit, donc des revues de code plus longues et plus difficiles.
- Une confiance aveugle dans le code proposé.
- Des failles classiques : injection SQL, XSS, secrets écrits en dur.
- Des dépendances obsolètes, voire inventées : un attaquant peut publier un paquet portant le nom inventé par l'IA (*slopsquatting*, une forme de Supply Chain Attack).

Les bonnes pratiques :

- Relire le code de l'IA comme celui d'un développeur débutant : vérifier chaque dépendance ajoutée et chaque requête écrite à la main.
- Donner des consignes de sécurité dans le prompt.
- Centraliser les règles de l'équipe dans un fichier `AGENTS.md` à la racine du projet. Ce fichier est lu automatiquement par les agents IA et évite de compter sur chaque développeur pour répéter les consignes.
- Garder les outils automatiques (SonarQube, Dependabot, etc.).

```
<!-- Exemple d'extrait de AGENTS.md -->
## Sécurité

- Ne jamais concaténer une donnée utilisateur dans une requête SQL : utiliser Eloquent ou des requêtes préparées.
- Ne jamais écrire de secret en dur : utiliser le fichier `.env`.
- Ne pas ajouter de dépendance sans l'indiquer explicitement.
```

> **Un bon prompt ne remplace pas la revue de code**
>
> L'IA peut ignorer une consigne, se tromper ou inventer une dépendance. **Vous restez responsable du code que vous poussez**, quel que soit l'outil qui l'a écrit.

Open Web Application Security Project (OWASP)

> Open Web Application Security Project (OWASP) est une communauté en ligne travaillant sur la sécurité des applications Web. Sa philosophie est d'être à la fois libre et ouverte à tous. Elle a pour vocation de publier des recommandations de sécurisation Web et de proposer aux internautes, administrateurs et entreprises des méthodes et outils de référence permettant de contrôler le niveau de sécurisation de ses applications Web.
>
> *Source: Wikipédia*

Site d'OWASP

Au passage

Depuis 2023, l'acronyme OWASP signifie officiellement « Open **Worldwide** Application Security Project », pour refléter un périmètre plus large que le seul Web.

Un changement récent

Le Top 10 vient d'être mis à jour : la version 2025 (8e édition) a été publiée fin 2025, la précédente datait de 2021. Vous croiserez donc encore beaucoup de ressources, d'articles et de sujets qui référencent la version 2021 : les codes et l'ordre changent, mais les grandes idées restent les mêmes.

OWASP liste 10 grandes catégories de failles **à connaître** (version 2025, la 8e édition du Top 10) :

- **A01:2025 – Contrôles d'accès défaillants** : Les vulnérabilités liées aux contrôles d'accès permettent aux utilisateurs d'agir en dehors de leurs permissions prévues. Cette catégorie inclut désormais la falsification de requête côté serveur (SSRF) : inciter le serveur à envoyer des requêtes vers des destinations non prévues (services internes, cloud metadata, etc.). Elle comprend aussi le CSRF et les redirections non validées.
- **A02:2025 – Mauvaise configuration de sécurité** : Configuration incomplète ou trop permissive, services inutiles activés, comptes par défaut non modifiés, messages d'erreur trop détaillés. En forte progression : de la 5e à la 2e place.
- **A03:2025 – Défaillances de la chaîne d'approvisionnement logicielle (Software Supply Chain Failures)** : Compromission des dépendances, des outils de build ou de la chaîne de distribution du logiciel. Élargit l'ancienne catégorie « Composants vulnérables et obsolètes » : c'est la traduction dans le Top 10 des Supply Chain Attacks vues plus haut.
- **A04:2025 – Défaillances cryptographiques** : Les données en transit et au repos (telles que les mots de passe, numéros de carte bleue, dossiers médicaux, informations personnelles et secrets commerciaux) requièrent une protection supplémentaire. Exemple : mots de passe non hachés en base de données, absence de HTTPS.
- **A05:2025 – Injection** : Les attaques par injection (SQL, NoSQL, OS, LDAP, ainsi que le XSS) surviennent lorsque des données non fiables sont envoyées à un interpréteur en tant que commande ou requête.
- **A06:2025 – Conception non sécurisée** : Désigne l'absence ou la faiblesse de la conception des contrôles de sécurité. Exemple : absence de contrôle dans un système de routage Web, manque de validation des entrées, absence de protection anti-bot.
- **A07:2025 – Défaillances d'authentification** : Vulnérabilités liées à l'authentification : absence de MFA, mots de passe faibles acceptés, sessions mal gérées, identifiants par défaut.
- **A08:2025 – Manque d'intégrité des données et du logiciel** : Code et infrastructure non protégés contre les violations d'intégrité. Exemple : mises à jour non signées, scripts chargés depuis un CDN sans vérification, pipelines CI/CD non sécurisés, désérialisation non sécurisée.
- **A09:2025 – Carence de journalisation et d'alerte** : Absence de logs, logs insuffisants, ou absence de surveillance et d'alerte permettant de détecter et répondre aux incidents de sécurité.
- **A10:2025 – Mauvaise gestion des conditions exceptionnelles** : Erreurs et cas limites mal gérés : exceptions non traitées, comportement « fail open » (le système laisse passer en cas d'erreur au lieu de bloquer), messages d'erreur qui révèlent des informations internes.

Top 10 : Simplifié

Le TOP 10 2025 est très intéressant, car il met en lumière le croisement entre les failles et les risques. Mais il est plus complexe à mémoriser. Il est donc également possible de classer les failles de manière brute :

- **Injection** : Injection SQL, NoSQL, OS, LDAP...
- **Violation de Gestion d'Authentification et de Session** : Risque de casser / usurper une authentification ou une session.
- **Cross-Site Scripting (XSS)** : Risque d'injection de contenu dans une page pour provoquer des actions non désirées.
- **Références directes non sécurisées à un objet** : Accès à de la donnée en spécifiant un `id` directement par un paramètre non filtré.
- **Mauvaise configuration Sécurité** : Failles liées aux serveurs Web, applications, base de données ou frameworks.
- **Exposition de données sensibles** : Exposition de données sensibles et nécessité de chiffrer ces données.
- **Manque de contrôle d'accès au niveau fonctionnel** : Failles liées aux contrôles d'accès de fonctionnalité.

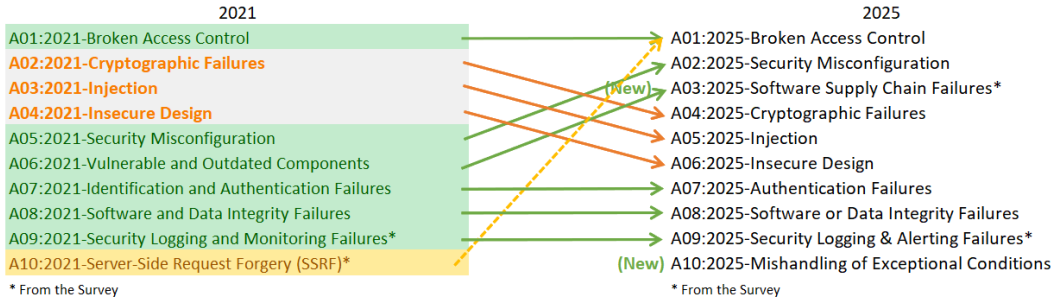
- **Falsification de requête intersite (CSRF)** : Failles liées à l'exécution de requêtes à l'insu de l'utilisateur.
- **Utilisation de composants avec des vulnérabilités connues** : Failles liées à l'utilisation de composants tiers vulnérables.
- **Redirections et Renvois non validés** : Vulnérabilité permettant de rediriger l'utilisateur sur une page malveillante.

Ce classement est plus simple à mémoriser et permet de se rappeler des failles les plus courantes.

Je n'invente rien

Ce classement correspond à la version OWASP TOP 10 de 2017. Bien que le classement officiel ait été mis à jour depuis (2021, puis 2025), ces concepts restent valables et pertinents.

OWASP, un classement qui évolue



Source

Qu'est-ce qui change par rapport à 2021 ?

La version 2025 (publiée fin 2025) apporte trois évolutions principales : le **SSRF** (ancien A10) est fusionné dans les contrôles d'accès (A01), la **chaîne d'approvisionnement logicielle** devient une catégorie à part entière (A03, élargissement de l'ancien « Composants vulnérables et obsolètes »), et la **mauvaise gestion des conditions exceptionnelles** fait son entrée (A10). La « Mauvaise configuration de sécurité » grimpe de la 5e à la 2e place.

Les failles et leurs corrections

Le Top 10 OWASP nous donne les grandes catégories de failles à connaître. Pour entrer dans le détail, voici pour chacune un exemple de code vulnérable et sa correction.

A01 : Contrôles d'accès défaillants

Accès à de la donnée ou à une fonctionnalité sans en avoir le droit. C'est quelque chose de très courant : par exemple, un `id` passé directement en paramètre et non contrôlé.

```php
// http://exemple.com/facture.php?id=42
// Et si je remplace 42 par 43 ?
$stmt = $pdo->prepare("SELECT * FROM facture WHERE id = ?");
$stmt->execute([$_GET["id"]]);
```

Requête préparée

Vous noterez ici que nous avons une requête « préparée », ça n'empêche pas le danger…

Autres exemples :

- Page d'admin accessible avec un compte utilisateur.
- Mode non filtré (`http://exemple.com/liste?mode=client`, et si je change client par admin ?).

```php
// admin.php : aucun contrôle du rôle
deleteUser($_GET['id']);
```

Comment corriger ?

- Toujours valider les entrées utilisateurs.
- Toujours vérifier les droits de l'utilisateur.

```
// La facture doit appartenir à l'utilisateur connecté
$stmt = $pdo->prepare("SELECT * FROM facture WHERE id = ? AND client_id = ?");
$stmt->execute([$_GET["id"], $_SESSION["client_id"]]);
```

```
// Une page d'administration vérifie toujours le rôle
if ($_SESSION['role'] !== 'admin') {
    die("Accès refusé.");
}
```

A01 : Falsification de requête intersite (CSRF)

Failles liées à l'exécution de requêtes à l'insu de l'utilisateur. Un site tiers peut envoyer le même formulaire que le vôtre, avec la session de l'utilisateur connecté.

```
<!-- Un site tiers peut envoyer le même formulaire -->
<form method="post" action="supprimer-compte.php">
    <button>Supprimer mon compte</button>
</form>
```

```
<!-- En GET, une simple image dans un mail suffit -->
<img src="http://exemple.com/supprimer-compte.php">
```

Comment corriger ?

Une action qui modifie des données passe toujours en POST. Ajoutez ensuite un jeton unique dans les formulaires. Celui-ci est intégré de base dans Laravel.

Génération du jeton (au moment d'afficher le formulaire) :

```
$_SESSION['token'] = bin2hex(random_bytes(32));
```

Dans le formulaire HTML :

```
<input type="hidden" name="token" value="<?= $_SESSION['token'] ?>">
```

Côté PHP, à la réception du formulaire :

```
if (!hash_equals($_SESSION['token'], $_POST['token'] ?? '')) {
    die("Requête invalide.");
}
```

Pourquoi `hash_equals()` et pas `==` ?

Une comparaison classique (`==` ou `===`) s'arrête au premier caractère différent. Le temps de réponse du serveur varie donc selon le nombre de caractères corrects au début de la chaîne. En mesurant très précisément ces temps de réponse, un attaquant peut deviner le jeton caractère par caractère : c'est une **attaque temporelle** (timing attack).

`hash_equals()` compare toujours les deux chaînes en entier, quel que soit l'endroit où elles diffèrent. Le temps de réponse est constant et ne révèle aucune information. À utiliser dès que vous comparez un secret (jeton CSRF, signature, clé API).

A01 : Redirections non validées

Les redirections non validées sont une vulnérabilité profitant d'une faiblesse dans le code et dont l'objectif est de rediriger l'utilisateur sur une page malveillante. Votre site sert alors de « masque » dans du phishing.

```
// http://www.shop-vdt.com/login.php?goto=evil.com/login
header("Location: " . $_GET['goto']);
```


Comment corriger ?

- Toujours valider les entrées utilisateurs.
- Filtrer les liens possibles.

```
// Uniquement des destinations connues
$whitelist = ['accueil', 'compte', 'panier'];

if (in_array($_GET['goto'], $whitelist)) {
    header("Location: index.php?page=" . $_GET['goto']);
}
```

A02 : Mauvaise configuration de sécurité

Correspond aux failles de configuration liées aux serveurs Web, applications, base de données ou frameworks.

- Console d'administration disponible sans authentification en ligne.
- Listage des répertoires (Exemple)
- Exemples de code non supprimés.
- Application en debug.

```
# Apache : la liste des fichiers est visible par tous
Options +Indexes
```

```
; PHP : les erreurs détaillées sont affichées aux visiteurs
display_errors = On
```

Comment corriger ?

- Toujours supprimer les exemples de code.
- Toujours supprimer les répertoires de débug.
- Lire la documentation.

```
Options -Indexes
```

```
display_errors = Off
log_errors = On
```

A03 : Chaîne d'approvisionnement logicielle

Failles liées à l'utilisation de composants tiers vulnérables ou compromis (voir les Supply Chain Attacks).

- CMS non à jour.
- Apache / Tomcat non patchés.
- Librairies XYZ non à jour.
- Version de PHP non à jour.
- Framework non à jour.

```
# Fichiers lock ignorés, dernières versions installées sans contrôle
composer update
npm update
```

```
{ "require": { "monolog/monolog": "*" } }
```

Comment corriger ?

- Toujours mettre à jour les composants tiers, en contrôlant ce qui est installé.
- Ne pas utiliser de vieux frameworks (exemple : PHP 4 ou Symfony 1.4).

```
# Fichiers lock versionnés : les mêmes versions partout
composer install
npm ci
```

```
# Recherche des vulnérabilités connues
composer audit
npm audit
```

```
{ "require": { "monolog/monolog": "^3.0" } }
```

A04 : Défaillances cryptographiques

Exposition de données sensibles comme les mots de passe, les numéros de carte de paiement ou encore les données personnelles et la nécessité de chiffrer ces données.

- Espace client sans HTTPS.
- Mot de passe en clair (ou en MD5) dans la base de données.
- Sauvegarde de données inutiles.
- Données sensibles dans les logs.
- Données sensibles en clair dans la base de données.

```
// En clair dans la base
$hash = $_POST['password'];

// MD5, SHA1 : beaucoup trop rapides à casser
$hash = md5($_POST['password']);
$hash = sha1($_POST['password']);
```

```
<!-- Sans HTTPS, le mot de passe circule en clair -->
<form method="post" action="http://exemple.com/login.php">
```

Comment corriger ?

- Toujours utiliser le HTTPS.
- Toujours stocker les mots de passe hachés et salés (jamais chiffrés, le hachage n'est pas réversible).
- Toujours supprimer les données inutiles.
- Toujours supprimer les données sensibles des logs.
- Protéger les données sensibles dans la base de données (chiffrement).

```
// À l'inscription
$hash = password_hash($_POST['password'], PASSWORD_DEFAULT);

// À la connexion
password_verify($_POST['password'], $hash);
```

```
<form method="post" action="https://exemple.com/login.php">
```

A05 : Injection SQL et Shell

Injection SQL, Shell... Souvent la plus connue et la plus rencontrée :

```
// http://exemple.com/liste?id=' OR '1'='1
$sql = "SELECT * FROM client WHERE id='" . $_GET["id"] . "'";
$result = $pdo->query($sql);
```

```
// dossier = ; cat /etc/passwd
system("ls " . $_GET['dossier']);
```


C'est la base de la sécurité

Vous trouverez cet exemple un peu partout. C'est le mauvais exemple en termes de sécurité !

Au passage, si vous écrivez :

```
$id = $_GET['id'];
$sql = "SELECT * FROM client WHERE id='{$id}'";
```

⚠️ C'est aussi une faille, celle-ci est identique à la précédente.

Comment corriger ?

- Toujours utiliser des requêtes préparées.
- Ou utiliser des ORM (Object Relational Mapping) qui font la même chose.

```
$stmt = $pdo->prepare("SELECT * FROM client WHERE id = ?");
$stmt->execute([$_GET["id"]]);
```

```
system("ls " . escapeshellarg($_GET['dossier']));
```

A05 : Cross-Site Scripting (XSS)

Risque d'injection de contenu dans une page dans le but de provoquer des actions non désirées dans celle-ci. Les failles XSS sont particulièrement répandues parmi les failles de sécurité Web.

Exécution de code JavaScript sans validation. Le risque ici est qu'il est possible de changer le comportement initialement attendu pour en détourner le sens.

```
// nom = <script>alert('XSS')</script>
echo "Bonjour " . $_POST['nom'];
```

Le code sera exécuté dans le navigateur de l'utilisateur lors de l'affichage de la page.

Deux types sont à connaître :

- XSS Persistant (stocké en base de données, dans un log, et exécuté à chaque affichage de la page)
- XSS Reflété (via un lien)

Comment corriger ?

Toujours échapper les données au moment de l'affichage :

```
echo "Bonjour " . htmlspecialchars($_POST['nom'], ENT_QUOTES, 'UTF-8');
```

D'autres fonctions existent pour filtrer les entrées :

```
$nom = filter_input(INPUT_POST, 'nom', FILTER_SANITIZE_FULL_SPECIAL_CHARS);
// ou (insuffisant seul : ne protège pas dans un contexte d'attribut HTML)
$nom = strip_tags($_POST['nom']);
```

Note importante

`FILTER_SANITIZE_STRING` est **déprécié depuis PHP 8.1**. Utilisez `FILTER_SANITIZE_FULL_SPECIAL_CHARS` ou `htmlspecialchars()`.

A06 : Conception non sécurisée

Application pensée sans prendre en compte la sécurité : aucune correction de code ne rattrape une mauvaise idée de départ. Par exemple une question secrète, avec peu de réponses possibles (peu de couleurs, peu de dates, etc.), ou l'absence d'anti-bots.

```
// Peu de réponses possibles : facile à deviner
if ($_POST['reponse'] == $user['couleur_preferee']) {
    resetPassword($user);
}
```

```php
// Aucun anti-bots : des milliers de comptes créés en boucle
createAccount($_POST['email']);
```

Comment corriger ?

- Penser la sécurité dès la conception.
- Remplacer la question secrète par un lien unique et temporaire.
- Ajouter un anti-bots (captcha) sur les formulaires sensibles.

```php
// Un lien unique et temporaire, envoyé par mail
$token = bin2hex(random_bytes(32));
saveResetToken($user, hash('sha256', $token), time() + 3600);
sendResetMail($user, $token);
```

```php
// Un captcha, vérifié côté serveur
if (verifyCaptcha($_POST['g-recaptcha-response'])) {
    createAccount($_POST['email']);
}
```

A07 : Défaillances d'authentification

Risque de casser / usurper une authentification ou une session. Comprend notamment le vol de session ou la récupération de mots de passe.

```php
// Aucune limite : des milliers d'essais possibles
if (password_verify($_POST['password'], $user['password'])) {
    $_SESSION['user_id'] = $user['id'];
}
```

Une session en paramètre GET == ⚠️. Si vous partagez le lien, n'importe qui pourra obtenir votre accès !

```
http://exemple.com/?jsessionid=A2938298D293
```

Comment corriger ?

- Limiter le nombre d'essais.
- Changer l'identifiant de session à la connexion.
- Toujours transmettre l'identifiant de session via un cookie sécurisé (`HttpOnly`, `Secure`), jamais dans l'URL.
- Toujours utiliser des sessions avec un identifiant unique et aléatoire.
- Toujours utiliser des sessions avec un TTL (Time To Live).

```php
if (tooManyAttempts($user)) {
    die("Compte temporairement bloqué.");
}

if (password_verify($_POST['password'], $user['password'])) {
    session_regenerate_id(true); // Nouvel identifiant de session
    $_SESSION['user_id'] = $user['id'];
}
```

```
; L'identifiant de session reste dans un cookie
session.use_only_cookies = 1
```

A08 : Manque d'intégrité des données et du logiciel

Du code ou des données utilisés sans vérifier qu'ils n'ont pas été modifiés. Par exemple un script chargé depuis un CDN, ou une mise à jour non signée.

```html
<!-- Et si le fichier est modifié sur le CDN ? -->
<script src="https://cdn.exemple.com/lib.js"></script>
```



```php
// Le cookie est modifiable par le visiteur
$panier = unserialize($_COOKIE['panier']);
```

Comment corriger ?

- Vérifier l'empreinte ou la signature de ce qui est chargé.

```html
<!-- Le navigateur vérifie l'empreinte du fichier -->
<script src="https://cdn.exemple.com/lib.js"
        integrity="sha384-..."
        crossorigin="anonymous"></script>
```

```php
// JSON : aucune création d'objet, et le contenu reste à valider
$panier = json_decode($_COOKIE['panier'], true);
```

A09 : Carence de journalisation et d'alerte

Sans logs, une attaque passe inaperçue.

```php
if (!password_verify($_POST['password'], $user['password'])) {
    // Personne n'est au courant de la tentative
    return false;
}
```

Comment corriger ?

- Journaliser les événements de sécurité (sans données sensibles).
- Surveiller ces logs et déclencher des alertes.

```php
if (!password_verify($_POST['password'], $user['password'])) {
    $ip = $_SERVER['REMOTE_ADDR'];
    error_log("Échec de connexion : compte {$user['id']}, IP $ip");
    return false;
}
```

A10 : Mauvaise gestion des conditions exceptionnelles

Que se passe-t-il quand ça se passe mal ? Ici, le système laisse passer en cas d'erreur (« fail open »).

```php
try {
    $isAdmin = checkAdmin($user);
} catch (Exception $e) {
    // Une erreur et tout le monde devient admin
    $isAdmin = true;
}
```

```php
} catch (PDOException $e) {
    echo $e->getMessage(); // Détails internes affichés au visiteur
}
```

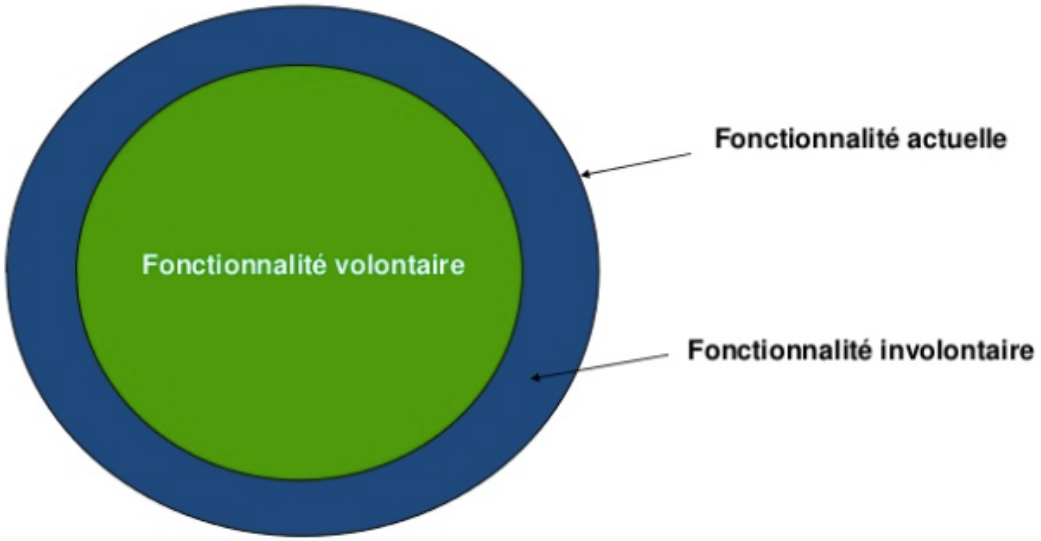
Comment corriger ?

- En cas d'erreur, toujours refuser l'accès (« fail closed »).
- Ne pas afficher le détail de l'erreur au visiteur, le garder dans les logs.

```php
try {
    $isAdmin = checkAdmin($user);
} catch (Exception $e) {
    // En cas d'erreur, on refuse l'accès
    error_log($e->getMessage());
    $isAdmin = false;
}
```

```php
} catch (PDOException $e) {
    error_log($e->getMessage());
    die("Une erreur est survenue.");
}
```

Mais, une faille c'est quoi ?



L'idée d'OWASP, c'est de former pour comprendre les failles afin de ne plus les produire involontairement… Et surtout avec OWASP on parle de **vulnérabilité, et non de risque**.

La gestion des secrets

Ne jamais écrire en dur

Les clés API, mots de passe de BDD, tokens et certificats ne doivent **jamais** figurer dans le code source.

- **Variables d'environnement** : Utiliser un fichier `.env` en local (hors git) et des variables d'environnement sur le serveur.
- **Ne pas versionner** : Ajouter `.env` au `.gitignore`. Un secret commité est un secret exposé.
- **Rotation** : Changer régulièrement les mots de passe et clés API en cas de fuite.
- **Vaults** : Pour les projets avancés, utiliser un coffre-fort (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault).

```php
// Mauvais
$apiKey = "sk-1234567890abcdef";

// Bon
$apiKey = getenv('API_KEY');
// ou via une librairie comme vlucas/phpdotenv
$apiKey = $_ENV['API_KEY'];
```

Vérifier avant de commit

Utilisez des outils comme `git-secrets`, `GitLeaks` ou `TruffleHog` pour scanner l'historique Git.

Sécuriser les API et les tokens

Quelques règles d'or quand on expose ou consomme une API :

- **JWT** : Ne stockez jamais d'informations sensibles dans le payload (il est juste encodé en Base64). Signez-le avec une clé forte, définissez une expiration courte et utilisez des refresh tokens.
- **Rate Limiting** : Protégez vos endpoints d'authentification et d'API contre le brute-force (ex. `throttle` de Laravel, middleware Express).
- **CORS** : Ne laissez jamais `Access-Control-Allow-Origin: *` sur une API privée. Restreignez aux domaines autorisés.
- **Ne pas passer de secrets en GET** : Les paramètres d'URL restent dans l'historique du navigateur et les logs du serveur.

```php
// Mauvais : token dans l'URL
// https://api.exemple.com/user?token=eyJhbGci...

// Bon : token dans le header Authorization
$headers = getallheaders();
$token = str_replace('Bearer ', '', $headers['Authorization'] ?? '');
```

Stocker un JWT

Ne stockez pas un JWT dans le `localStorage` si vous pouvez l'éviter (risque XSS). Préférez un cookie `HttpOnly` / `Secure`.

Les headers de sécurité HTTP

Les headers HTTP sont le premier rempart côté navigateur. Ils se configurent côté serveur.

Header	Rôle
`Content-Security-Policy` (CSP)	Empêche l'exécution de scripts inline et limite les sources de contenu (protection XSS).
`Strict-Transport-Security` (HSTS)	Force le navigateur à n'utiliser que HTTPS.
`X-Frame-Options`	Empêche le clickjacking (ex. `DENY`, `SAMEORIGIN`).
`X-Content-Type-Options`	Empêche le navigateur de deviner le type MIME (`nosniff`).
`Referrer-Policy`	Contrôle la fuite d'information dans l'en-tête `Referer`.

```php
// Exemple en PHP
header("Content-Security-Policy: default-src 'self'; script-src 'self'");
header("Strict-Transport-Security: max-age=31536000; includeSubDomains");
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("Referrer-Policy: strict-origin-when-cross-origin");
```

Les cookies sécurisés

Quand vous posez un cookie de session :

- `HttpOnly` : Empêche JavaScript d'y accéder (protection XSS).
- `Secure` : N'envoie le cookie que sur HTTPS.
- `SameSite=Strict` (ou `Lax`) : Protège contre le CSRF en ne l'envoyant pas depuis un autre site.

```php
setcookie('session', $token, [
    'expires' => time() + 3600,
    'path' => '/',
    'secure' => true,
    'httponly' => true,
```

```
    'samesite' => 'Strict'
]);
```

Les outils OWASP

- OWASP Juice Shop (Formation, JavaScript)
- WebGoat (Formation, Java)
- ZAP - Zed Attack Proxy (audit ; a quitté la fondation OWASP en 2023, désormais porté par Checkmarx)
- OWASP Testing Guide (Guide de test de sécurité)
- OWASP Code Review Guide (Méthode d'audit de code)
- OWASP Dependency-Check (Vérification des composants vulnérables)

DevSecOps et CI/CD sécurisé

La sécurité ne se fait pas qu'en production. Il faut l'intégrer dès la phase de développement (*shift-left*).

- **Analyse statique (SAST)** : SonarQube, Semgrep, CodeQL analysent le code à la recherche de failles sans l'exécuter.
- **Scan de dépendances (SCA)** : Snyk, Dependabot, OWASP Dependency-Check vérifient que vous n'utilisez pas de librairies vulnérables.
- **Scan de secrets** : GitLeaks ou TruffleHog dans la pipeline bloquent un commit contenant une clé API.
- **Tests de sécurité automatisés** : ZAP en mode headless dans votre CI pour détecter les régressions.

```yaml
# Exemple conceptuel de pipeline CI
stages:
  - build
  - test
  - security

sast:
  stage: security
  script:
    - sonar-scanner

dependency_check:
  stage: security
  script:
    - dependency-check.sh --project MonApp --scan .

secret_detection:
  stage: security
  script:
    - gitleaks detect --source .
```

Le pré-commit hook

Avant même la CI, bloquez les secrets en local :

```
# .git/hooks/pre-commit
gitleaks protect --staged
```

La formation

En cybersécurité, il est important de rappeler que la formation des employés est primordiale. Cette formation doit être :

- Régulière (tous les ans).
- Adaptée à l'entreprise (pas de formation générique).

- Prendre plusieurs formes (phishing fictif, formation en ligne, formation en présentiel).

La formation prend également la forme de **sensibilisation** :

- Affichage de consignes de sécurité.
- Sensibilisation aux risques.
- Formation aux bonnes pratiques.
- Formation sur les mots de passe.
- Chocoblast (technique pour rappeler aux utilisateurs l'importance de verrouiller leur session).